Lumido — Lighthouse 3D Reconstruction

Pixel-Ops Trial Round · Team: **Halo** · Dataset: “Lighthouse” (200 senseFly albris images, 7152×5368, 3 flights) · Code: `pipeline/step1…step6`, run with `./run_all.sh`

1. Approach / pipeline

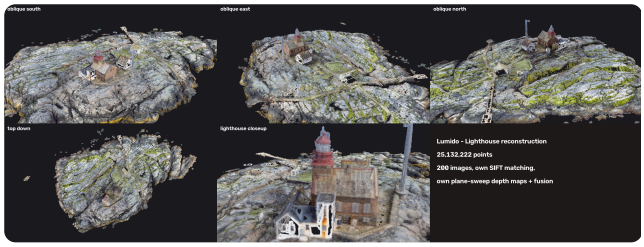
1. **Preparation** — resize to 1600 px (fits 8 GB RAM); read EXIF GPS + focal length; convert focal mm → px from sensor pixel density (1282 px).
2. **Features & matching (own code)** — OpenCV SIFT, 8000 keypoints/image, RootSIFT descriptors. GPS-guided pair selection (8,342 of 19,900 pairs: orbit images all-vs-all, others within 70 m). FLANN 2-NN + Lowe ratio 0.8 + mutual check + Essential-matrix RANSAC (1 px) → **3,044 verified pairs**.
3. **Structure-from-Motion** — our keypoints/matches loaded into pycolmap, used *as a backend library* for incremental registration (PnP) + bundle adjustment, refining focal length and radial distortion. Then a 7-DoF similarity fit of camera centres to GPS gives metric scale, Z up.
4. **Depth maps (own plane-sweep MVS)** — per image: undistort, pick 5 neighbours (most shared points, 3–45° viewing angle), sweep 160 fronto-parallel planes (uniform in 1/depth); each plane warps neighbours by the homography $H = K(R + t\,n^T/d)K^{-1}$; cost = 1 − ZNCC (7×7), mean of best 2 neighbours (occlusion-robust); winner-take-all + parabolic sub-plane refinement; reject low-texture / high-cost pixels; speckle filter removes small blobs (moving water).
5. **Fusion** — back-project every depth pixel to 3D, re-project into neighbours; keep points confirmed by ≥2 neighbours within 1 % depth, averaged over confirming views. Voxel merge (6 cm, batched), crop to survey area, statistical outlier removal → binary PLY.
6. **Rendering** — own NumPy z-buffer point renderer (look-at cameras) for screenshots; Open3D viewer for live demo.

2. Libraries / tools

- **OpenCV 5.0** — SIFT, FLANN, `findEssentialMat`, undistort, `warpPerspective`, box filters, `filterSpeckles`
- **pycolmap 4.2** — backend only: incremental mapper, bundle adjustment, GPS similarity alignment. *Not used:* COLMAP GUI, its feature extraction/matching or dense MVS.
- **NumPy** (all geometry, fusion, renderer) · **Open3D 0.20** (outlier removal, viewer) · **Pillow** (EXIF) · **Matplotlib** (plots) · Python 3.14 on an Apple M3 laptop (CPU only).

3. Results

Cameras registered	**200 / 200** in one model
Sparse points	~104,000, mean reprojection error **0.91 px**
Focal length	EXIF 1281.6 px → refined 1290.6 px (0.7 %)
GPS agreement	median **2.2 m** camera-to-GPS (drone GPS accuracy)
Depth maps	198 × (1200×900), in `outputs/depth_maps/`
Dense cloud	**25.1 M points**, 6 cm spacing, 261×156×36 m, `dense_cloud.ply`
Runtime	≈ 45 min total on laptop CPU



4. Known limitations

- **Sea is not reconstructed** — moving water breaks multi-view consistency; removed on purpose (texture/cost checks, speckle filter, fusion).
- **Vertical walls sparser than roofs/ground**; small holes under eaves and behind buildings — only 36 low oblique images see the façades.
- **Resolution trade-off** — depth computed at 1200×900 (~17 % of original pixels) to fit 8 GB RAM / CPU time; fine detail (railings, antenna) is thin.
- **Absolute position ±2 m** (consumer GPS); relative geometry is set by bundle adjustment (sub-pixel).
- 2 images got no depth map (fewer than 2 suitable neighbours); no mesh/texture step.